IncidentIQ

ECE532 - Final Presentation

Presenters:

Bence Suranyi
Darrian Shue
Richard Wu
Maanik Gogna

Agenda

01 Motivations & High Level Description

02 Initial Goals & Block Diagram

03 Updated Block Diagram

04 Explanation Of Major Components

05 System Demonstration

Motivations: *"Black Box" & Aviation Industry*

1) Crucial Role In Accident Reconstruction

- Currently used as crucial post accident evidence for incident reconstruction.
- Utilized to inform key policy changes.

2) Currently Only Used In Aircraft

- Potential applications in many other types of vehicles to record incidents.

3) Proprietary

- Current designs are proprietary and not open source.
- Cannot be adapted to other types of vehicles.

Meet: IncidentIQ

- Modern trends show growing popularity for telemetry as a form of insurance (i.e dashcams)

Capabilities:

- Collect telemetry data from a variety of sensors to allow for accident reconstruction.
- Gyroscope, accelerometer, temperature sensor, GPS.
- Remote logging over BT, failsafe onboard as SD card.

Initial Goals

Goal 1

Integrate all sensor functionality for purposes of data acquisition.

Goal 2

Develop a crash detection IP module that uses sensor data to determine when a vehicle has crashed (acceleration, angular speed, temperature)

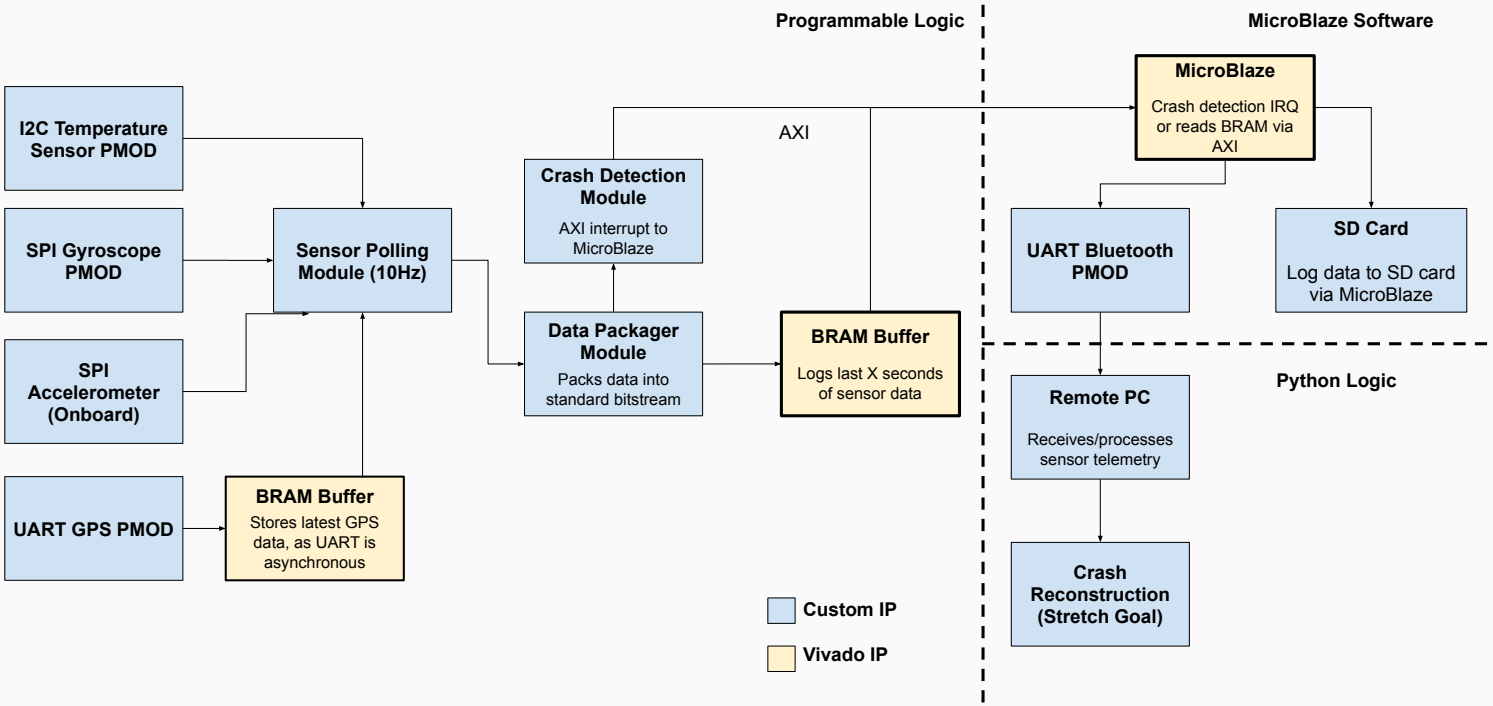
Goal 3

Transmit real time data (10Hz) over bluetooth to a user PC for logging, and save a recording of data on a locally installed SD card.

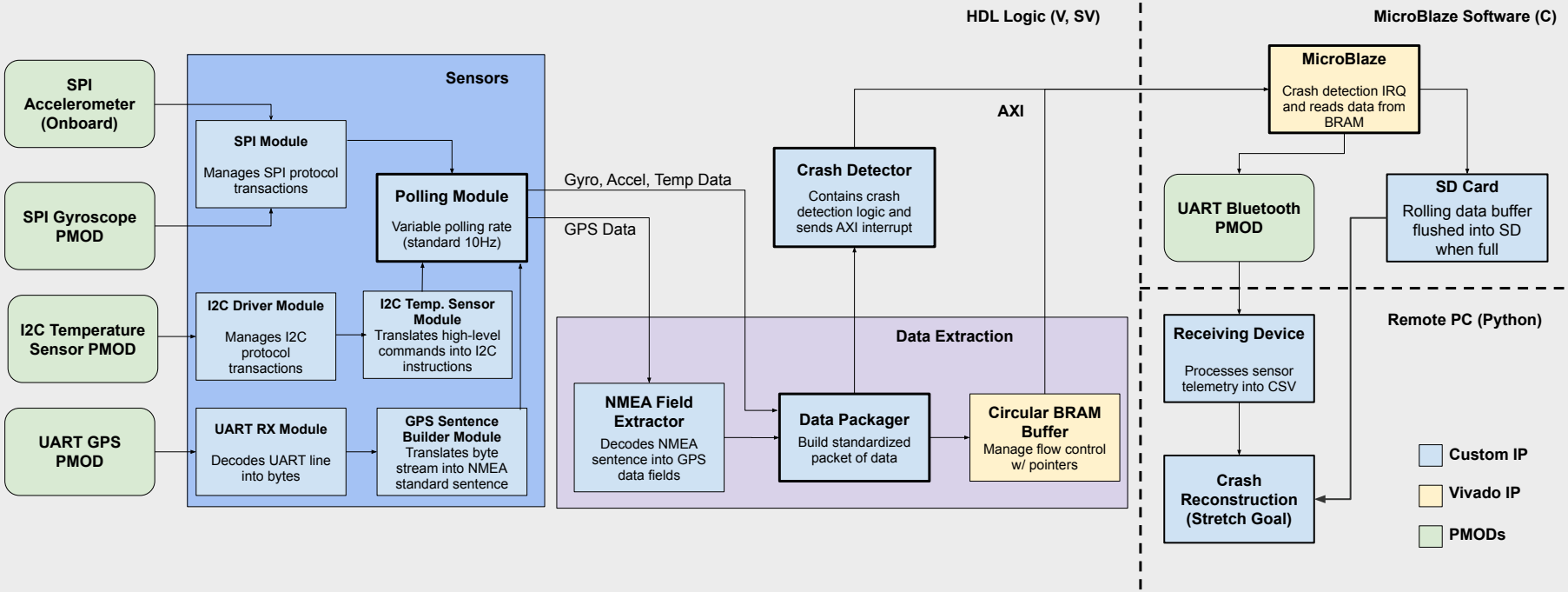
Stretch Goal

Development of a visualizer, to run on user PC and reconstruct an accident based on telemetry data.

Initial Block Diagram



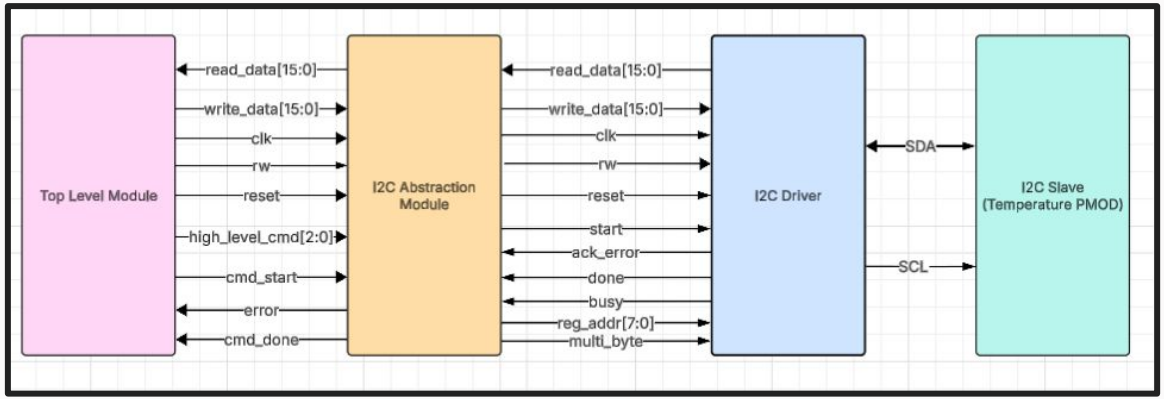
Final Block Diagram

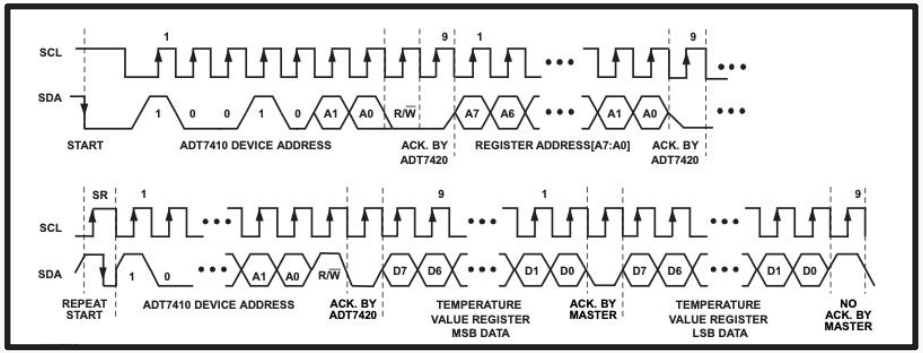


Protocols

I2C Driver and Temp Sensor Module

- I2C Driver capable of delivering I2C instructions to slaves (Temperature Sensor PMOD)
- Temp Sensor Abstraction Module abstracts PMOD-specific instructions and exposes high-level commands to polling module, reducing complexity at higher levels
- Capable of single- and multi-byte reads and writes as necessary for the Temperature Sensor PMOD



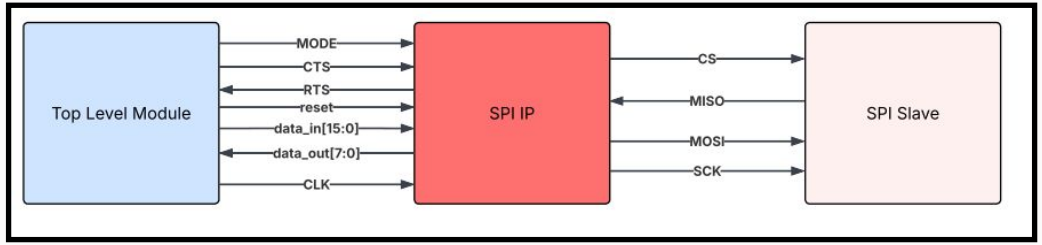


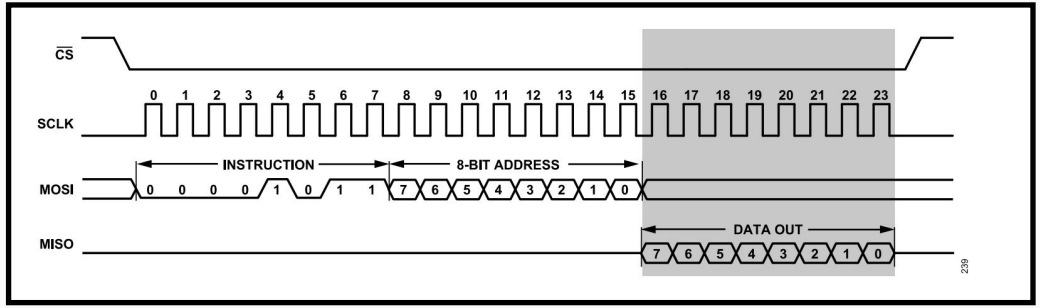
SPI Driver and Accelerometer / Gyro Modules

- SPI Driver capable of communicating with both accelerometer and gyro modules.

- SPI Driver can automatically configure accelerometer and gyro modules once enabled (abstracts this away from higher level modules).

- Capable of burst transfers.



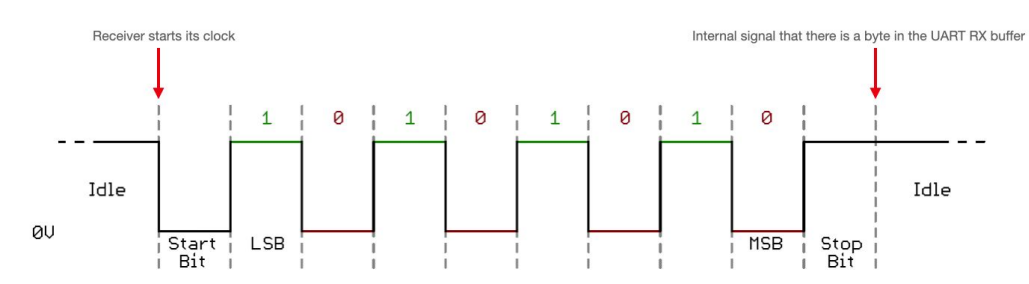


UART RX and GPS Decode Modules

- UART RX module extracts data from the TX pin on the GPS PMOD, converting each UART bitstream transmission into one byte of data

- Bytes formatted into National Marine Electronics Association (NMEA) standard ASCII GPS messages

- GPS sentence extractor filters for “$GPRMC” type messages, PMOD update rate is 1Hz



Example	Description
$GPRMC	Message ID
064951.000	UTC Time (hhmmss.sss)
A	Status (A = data valid)
2307.1256	Latitude (ddmm.mmmm)
N	N/S indicator
12016.4438	Longitude (dddmm.mmmm)
E	E/W indicator
0.03	Speed over ground (knots)
165.48	Course over ground (degrees)
260406	Date (ddmmyy)
3.05	Magnetic Variation (degrees)
W	E/W indicator
A	Mode (see GlobalTop manual)
*55	Checksum
<CR><LF>	End of message indicator

Table 5. RMC.

$GPRMC,064951.000,A,2307.1256,N,12016.4438,E,0.03,165.48,260406,3.05,W,A*55<CR><LF>

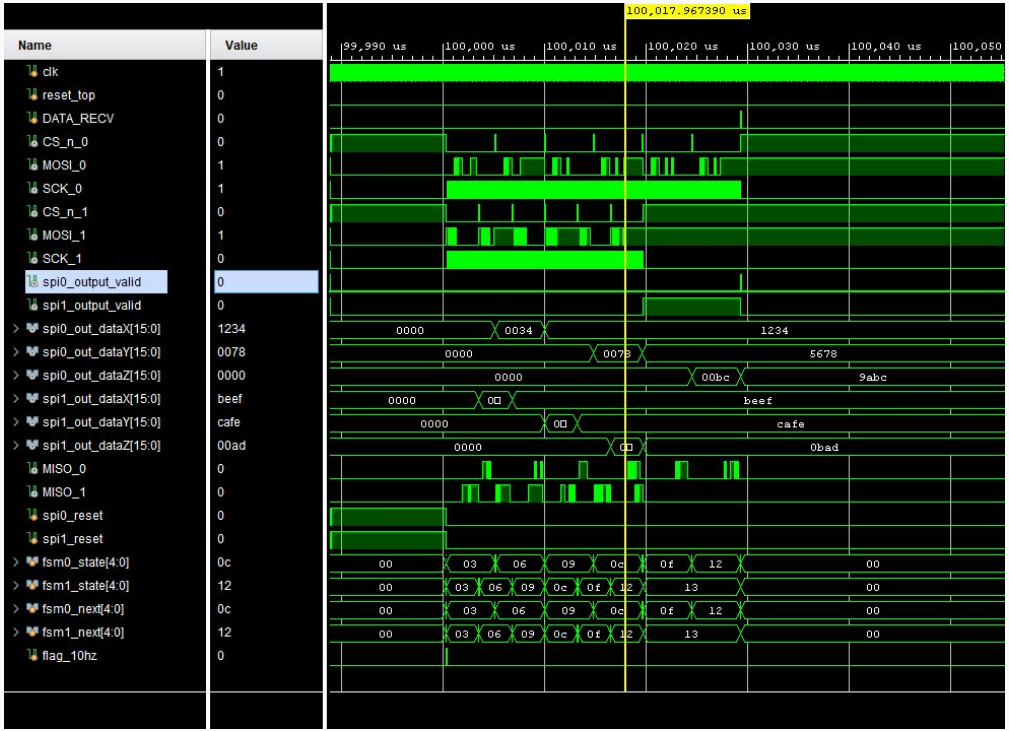
Processing

Sensor Polling Module

- Implements separate FSMs to manage the gyroscope sampling, accelerometer sampling, GPS sampling, and temperature sensor sampling.

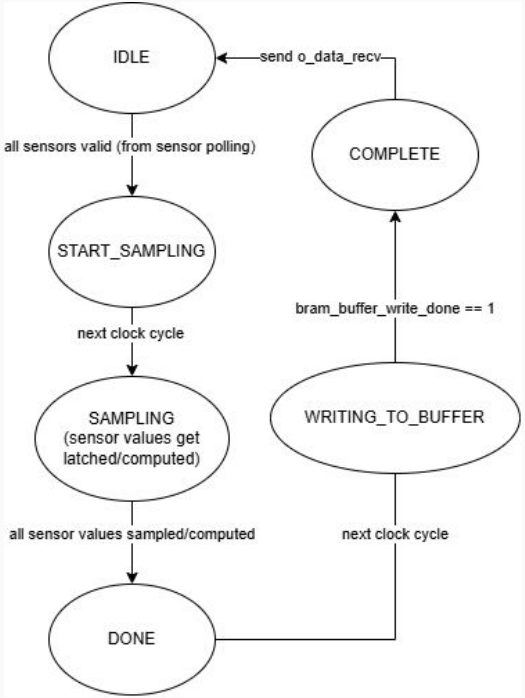
- Implements a timer to enforce new samples @10Hz, unless downstream pipeline stalls.

- Implements a valid/ready handshake with downstream data packager.



Data Packager Module

- Polls sensor data via control by the Sensor Polling Module
- Sensor data is organized into a 40 byte packet that is written to a BRAM buffer to be received by the MicroBlaze (more on this later)
- Only relevant GPS fields are included in the packet (extracted by decode module described previously)



Data packet structure (32-bit words)

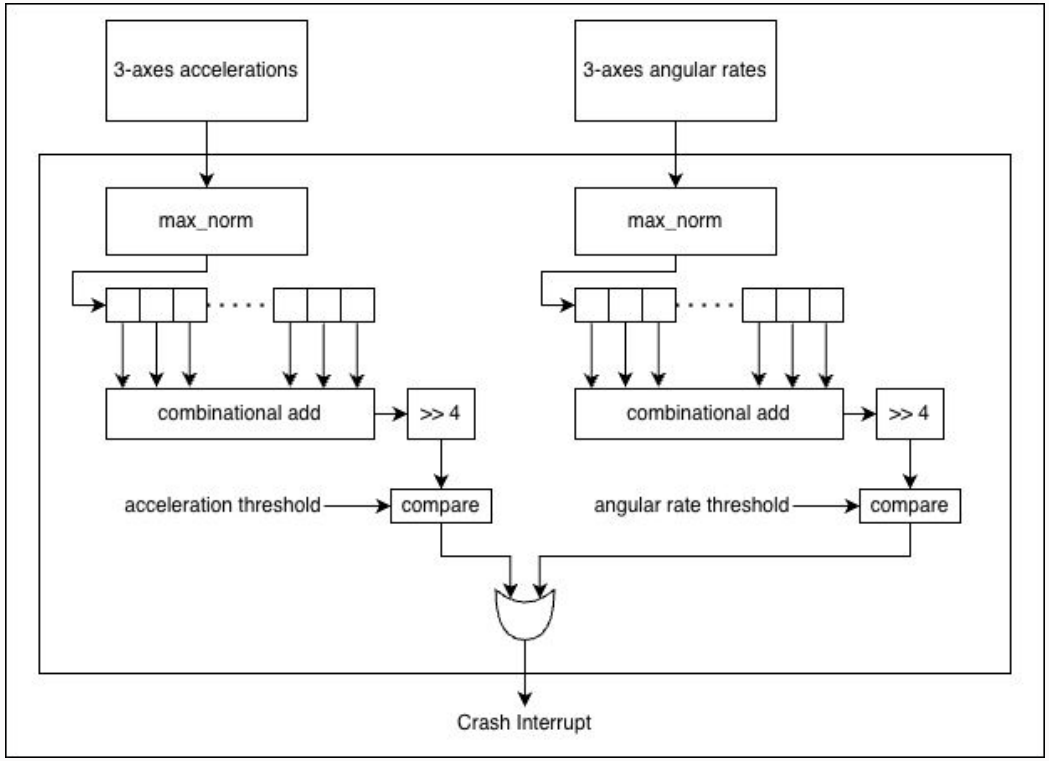
Temperature [15:0]
Gyro Y [63:48]; Gyro X [47:32]
Gyro Z [79:64]
Accel Y [127:112]; Accel X [111:96]
Accel Z [143:128]
GPS Ground Speed [191:160]
GPS N [193]; GPS E [192]
GPS Longitude [255:224]
GPS Latitude [287:256]
GPS UTC Time [319:288]

Crash Detection Module

- Saves a 16-frame history of acceleration and angular rate values and computes a running average

- The average acceleration and average angular rate are compared to configurable threshold values (more on this later)

- If thresholds are crossed, an interrupt is sent to MicroBlaze



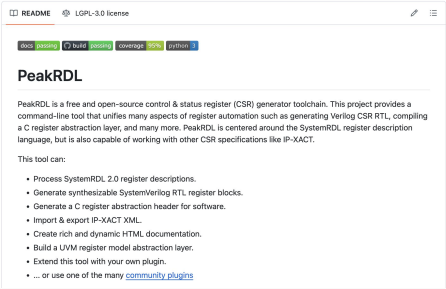
Firmware and Software

MicroBlaze Firmware: Memory

Data Packet Buffer:

- Uses 8KB BRAM as a circular FIFO buffer
- Data packager writes 10 32-bit words (40 bytes) for each packet (written over 10 cycles)
- MicroBlaze regularly reads out of the buffer
- Registers used to keep track of write and read positions

Configurable Registers: SystemRDL + [PeakRDL]


README LGPL-3.0 license

docs passing build passing coverage 95% python 3

PeakRDL

PeakRDL is a free and open-source control & status register (CSR) generator toolchain. This project provides a command-line tool that unifies many aspects of register automation such as generating Verilog CSR RTL, compiling a C register abstraction layer, and many more. PeakRDL is centered around the SystemRDL register description language, but is also capable of working with other CSR specifications like IP-XACT.

This tool can:

- Process SystemRDL 2.0 register descriptions.
- Generate synthesizable SystemVerilog RTL register blocks.
- Generate a C register abstraction header for software.
- Import & export IP-XACT XML.
- Create rich and dynamic HTML documentation.
- Build a UVM register model abstraction layer.
- Extend this tool with your own plugin.
- ... or use one of the many community plugins

```
addrmap registers {
    name = "Register Map";
    desc = "All-Purpose Registers";

    default hw = r;       // hardware can read by default
    default sw = rw;      // software can read/write by default

    /* ------------------ Data Packet Buffer ------------------ */
    // ---------------------------------------------------------------------
    // Data Packet Buffer Write Pointer Register
    // Written by custom IP hardware.
    // Read by MicroBlaze to determine how much data is available.
    // ---------------------------------------------------------------------
    reg data_packet_buffer_write_ptr_reg {
        name = "Write Poitner";
        desc = "Points to the next location the custom IP will write to.
                Valid range: 0 to 2047.
                Written by hardware, read by MicroBlaze.";

        // upper bits unused - buffer depth is 2048 words (11 bits needed)
        field {} RESERVED[31:11] = 0;
        field {
            name = "WPTR";
            desc = "Write pointer index (word address, 0-2047)";
            hw = w; // custom IP writes this
            sw = r; // MicroBlaze can only read it
        } WPTR[10:0] = 0; // reset to default of 0
    };

    // ---------------------------------------------------------------------
    // Data Packet Buffer Read Pointer Register
    // Written by MicroBlaze after consuming data.
    // Read by custom IP hardware to determine how much space is available.
    // ---------------------------------------------------------------------
    reg data_packet_buffer_read_ptr_reg {
        name = "Read Pointer";
        desc = "Points to the next location MicroBlaze will read from.
                Valid range: 0 to 2047.
                Written by MicroBlaze, read by hardware.";

        field {} RESERVED[31:11] = 0;
        field {
            name    = "RPTR";
            desc    = "Read pointer index (word address, 0-2047)";
            hw      = r;    // custom IP reads this
            sw      = rw;   // MicroBlaze reads and writes this
        } RPTR[10:0] = 0;
    };
```

MicroBlaze Firmware: Event Loop

- **Initialization** block programs registers to enable polling, data packaging, crash detection, and interrupts. Programs thresholds for accelerometer and gyroscope crash limits

- Also initializes SD card (mount / open file), and Bluetooth module. SD card filesystem operations are implemented using an open source library called FatFS

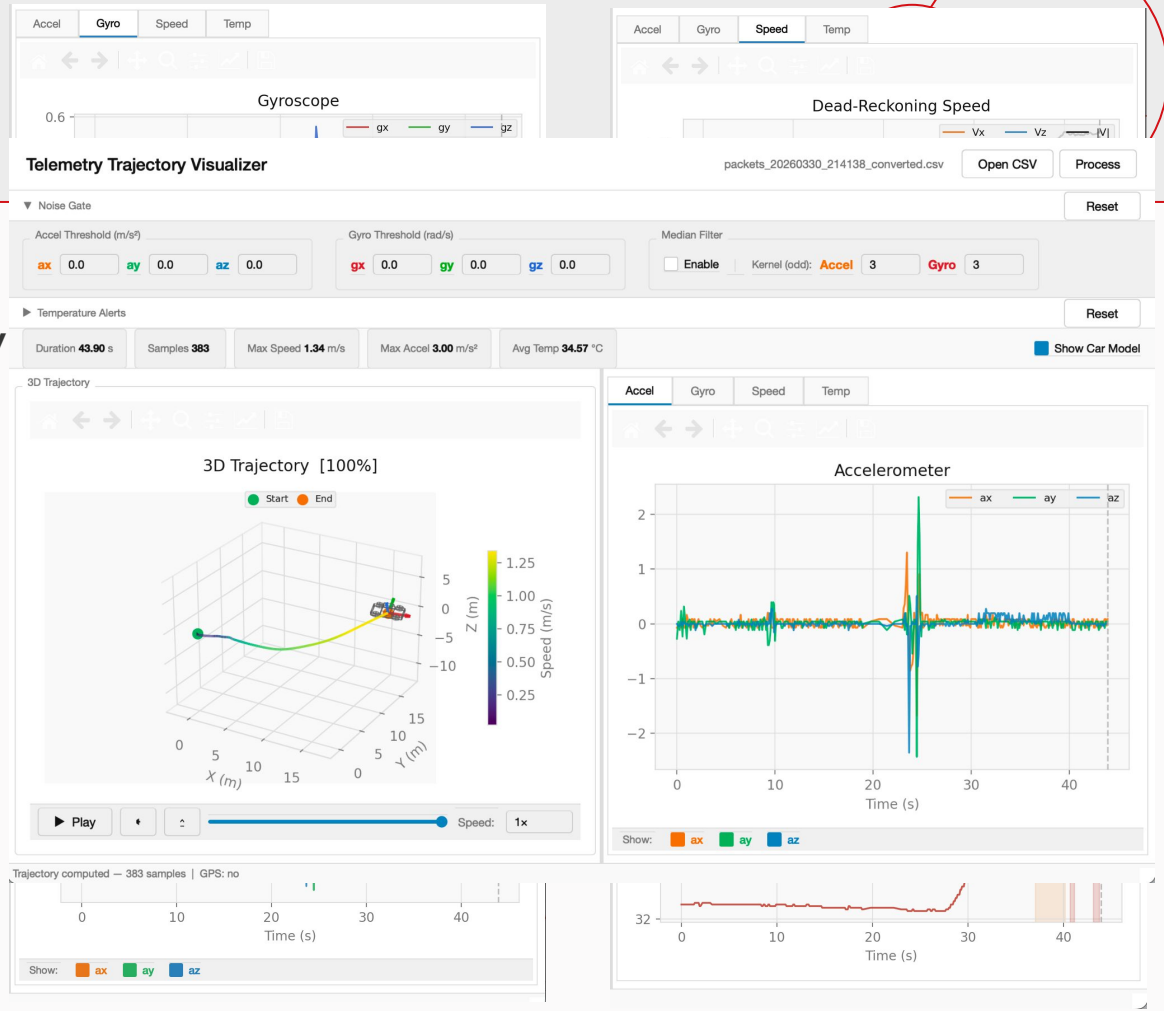
- **Main loop** compares read and write pointers to determine if new packet should be read. Read packet from BRAM, store into separate history buffer, and send live over Bluetooth to receiver

- If crash interrupt occurs, data packet is modified to flag a crash

- When the history buffer fills up, it is flushed and its contents are written to a file on the SD card

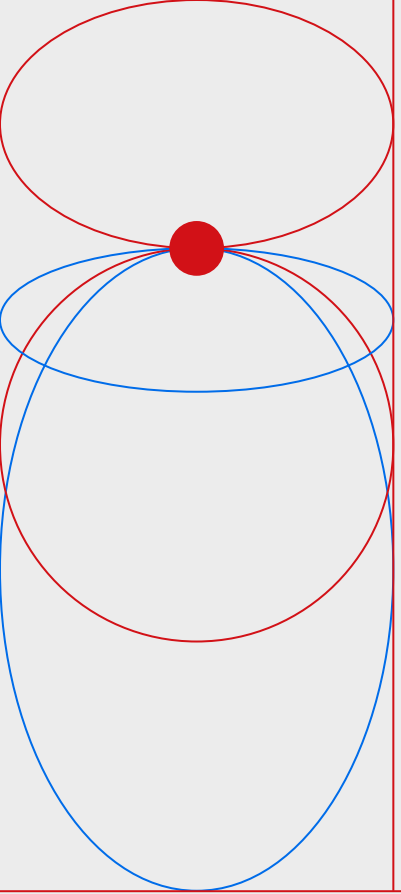
Data Visualizer

- Implementation of **3D reconstruction of crash trajectory**
- Capable of isolated representation of individual data metrics (accel., gyro. speed, temp.)
- Integration of **configurable thresholds** and smoothing filters for noisy data points (accel., gyro)
- Sensitive to temperature changes over time and in relation to static thresholds (configurable metrics)



Project Improvements / Next Steps

- **Crash Detection Module:** integrating jerk (integration of acceleration) to capture sudden-onset events that current averager misses
- **Bluetooth Transmission:** introducing data redundancy to reduce the chance of data packet loss during transmission (which could be probable during violent movements/crashes)
- **Visualizer:** flagging where a crash was detected could assist in viewing pre- and post-crash activity in relation to data reconstruction
- **Improved Customizability:** allowing thresholds to be set dynamically (ie. from the visualizer) which would then be communicated to the crash detection module over bluetooth during runtime, instead of requiring reflashed code each time



Demo

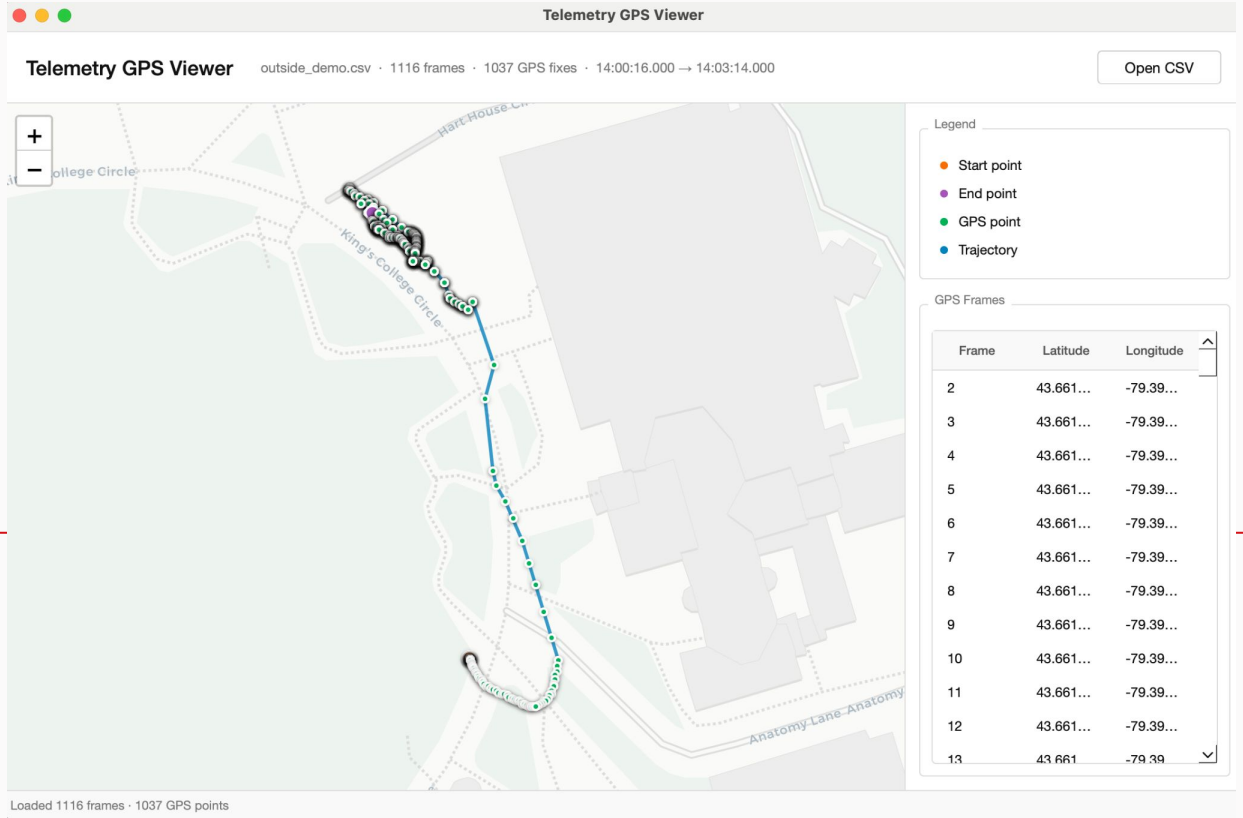
Crash Detection Demo (live)

1. Collect data
2. Do some motion
3. View data on Visualizer

GPS Demo

- Walked around Front Campus collecting data
- Data: **`outside_demo.csv`**
- Can see path on GPS viewer visualizer
- Video Link

GPS Demo


Telemetry GPS Viewer
Telemetry GPS Viewer
outside_demo.csv · 1116 frames · 1037 GPS fixes · 14:00:16.000 → 14:03:14.000
Open CSV
+
−
Hart House Cir
ollege Circle
King's College Circle
Anatomy Lane Anatomy
Legend
Start point
End point
GPS point
Trajectory
GPS Frames
Frame Latitude Longitude
2 43.661… -79.39…
3 43.661… -79.39…
4 43.661… -79.39…
5 43.661… -79.39…
6 43.661… -79.39…
7 43.661… -79.39…
8 43.661… -79.39…
9 43.661… -79.39…
10 43.661… -79.39…
11 43.661… -79.39…
12 43.661… -79.39…
13 43.661 -79.39
Loaded 1116 frames · 1037 GPS points

Temperature Change Demo

- Proxied a ‘crash’ that results in temperature rise
- Data: **`temp_rise_demo.csv`**
- Can view results in Visualizer
- Video Link